

Лабораторна робота №9

Тема: Розширені можливості Redis

З дисципліни: Базы даних та інформаційні системи
Студента групи МІТ-31: Дашковського М. О.

Хід роботи

1. Транзакції у Redis

Транзакції у Redis розпочинаються командою MULTI та дозволяють ставити декілька команд у чергу та виконувати їх командою EXEC. Для переривання транзакції варто скористатись командою DISCARD.

```
127.0.0.1:6379(TX)> SET counter 100
QUEUED
127.0.0.1:6379(TX)> INCR counter
QUEUED
127.0.0.1:6379(TX)> SET user "Max"
QUEUED
127.0.0.1:6379(TX)> EXEC
1) OK
2) (integer) 101
3) OK
```

Рисунок 9.1 – Транзакція в Redis

Команда WATCH дозволяє обрати ключ, що буде відстежуватись. Якщо його значення буде змінено то наступний EXEC не відбудеться.

```
127.0.0.1:6379> WATCH counter
OK
127.0.0.1:6379> MULTI
OK
127.0.0.1:6379(TX)> INCR counter
QUEUED
```

Рисунок 9.2 – Встановлення WATCH та початок транзакції

```
127.0.0.1:6379> DECR counter
(integer) 100
127.0.0.1:6379>
```

Рисунок 9.3 – Зміна ключа з іншого redis-cli

```
127.0.0.1:6379(TX)> EXEC
(nil)
```

Рисунок 9.4 – Транзакція не відбулась

2. Lua-скрипти в Redis

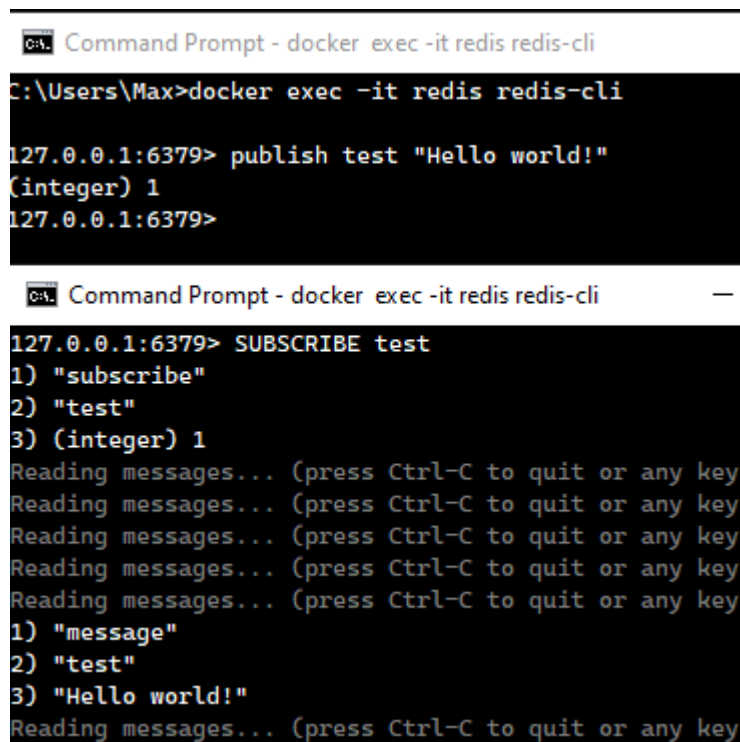
Команда EVAL у Redis дозволяє виконувати Lua-скрипти безпосередньо на сервері.

```
127.0.0.1:6379> EVAL "if redis.call('exists', KEYS[1]) == 0 then
return redis.call('set', KEYS[1], ARGV[1]) else return 'exists'
end" 1 testkey "value"
OK
127.0.0.1:6379> get testkey
"value"
```

Рисунок 9.5 – Використання EVAL для виконання Lua скрипта, що у випадку якщо ключ не існує створює його з заданим значенням

3. Механізм Pub/Sub

Механізм Pub/Sub (Publish/Subscribe) у Redis дозволяє організувати асинхронний обмін повідомленнями.



```
Command Prompt - docker exec -it redis redis-cli
C:\Users\Max>docker exec -it redis redis-cli
127.0.0.1:6379> publish test "Hello world!"
(integer) 1
127.0.0.1:6379>

Command Prompt - docker exec -it redis redis-cli
127.0.0.1:6379> SUBSCRIBE test
1) "subscribe"
2) "test"
3) (integer) 1
Reading messages... (press Ctrl-C to quit or any key)
Reading messages... (press Ctrl-C to quit or any key)
Reading messages... (press Ctrl-C to quit or any key)
Reading messages... (press Ctrl-C to quit or any key)
Reading messages... (press Ctrl-C to quit or any key)
1) "message"
2) "test"
3) "Hello world!"
Reading messages... (press Ctrl-C to quit or any key)
```

Рисунок 9.6 – Організація Pub/Sub у двох терміналах

4. Redis Streams (поток даних)

Redis Streams — це інструмент для обробки даних у реальному часі, схожий на журнали подій (event logs) або черги повідомлень. До потоку можна тільки додавати події в кінець — змінити додані події вже не вдасться.

```
127.0.0.1:6379> XADD mystream * sensor-id 1234 temperature 12.3
"1745698486117-0"
127.0.0.1:6379> _
```

Рисунок 9.7 – Додавання події до потоку

```
127.0.0.1:6379> publish test "Hello world!"
(integer) 1
127.0.0.1:6379> XRANGE mystream - +
1) 1) "1745698486117-0"
   2) 1) "sensor-id"
      2) "1234"
      3) "temperature"
      4) "12.3"
127.0.0.1:6379> XREAD COUNT 2 STREAMS mystream 0
1) 1) "mystream"
   2) 1) 1) "1745698486117-0"
      2) 1) "sensor-id"
         2) "1234"
         3) "temperature"
         4) "12.3"
127.0.0.1:6379> _
```

Рисунок 9.8 – Зчитування подій з потоку

5. (Додатково) Напишіть скрипт (Python/JS), який реалізує логіку Pub/Sub або читає дані зі Stream.

Для цього завдання було допрацьовано скрипт з минулої роботи. Тепер для відправки та зчитування повідомлень використовується механізм Pub/Sub

```
import redis, time

r = redis.Redis(host='localhost', port=6379, decode_responses=True)
channel = "messages_channel"
pubsub = r.pubsub()
pubsub.subscribe(channel)

for message in pubsub.listen():
    if message['type'] == 'message':
        print(message['data'])
```

Рисунок 9.9 – Код message_display.py

```
import redis

r = redis.Redis(host='localhost', port=6379, decode_responses=True)
channel = "messages_channel"

username = input("Enter your username: ")

while True:
    msg = input('> ')
    if msg!="":
        msg = f"{username}: {msg}"
        r.publish(channel, msg)
```

Рисунок 9.10 – Код messenger.py

```
C:\WINDOWS\py.exe
user2: Hello
user2: is anyone online?
user1: me
user2: cool, whatsup?
user1: i'm ok, hbu?
user2: i'm fine too
user2: any news?

C:\WINDOWS\py.exe
Enter your username: user1
> me
>
> i'm ok, hbu?
>

C:\WINDOWS\py.exe
Enter your username: user2
> Hello
> is anyone online?
> cool, whatsup?
> i'm fine too
> any news?
>
```

Рисунок 9.11 – Запущені скрипти

6. Теоретичне завдання

Визначте, які з нових механізмів Redis можуть бути корисними у великих розподілених системах.

У розподілених системах можна використовувати WATCH для запобігання конкурентним змінам (Optimistic Locking), а також Pub/Sub та потоки для обміну повідомленнями та розповсюдженням їх між окремими хостами.

Які плюси і мінуси Redis як брокера повідомлень?

До плюсів відносять швидкість, простоту, низьку затримку, підтримку Pub/Sub та потоків (streams). Мінусами є можливі втрати даних, відсутність гарантій доставки, обмеженість у складних сценаріях, високе споживання пам'яті.

Висновок: під час роботи було розглянуто розширені можливості Redis. Було розглянуто такі механізми як транзакції, Lua скрипти, Pub/Sub та streams (потоки).